
11. SEARCH ALGORITHMS

A. LINEAR AND BINARY SEARCH

PYTHON : STRING AND LIST METHODS

```
string = 'mississippi'
```

```
lst = ['m', 'i', 's', 's', 'i', 's', 's', 'i', 'p', 'p', 'i']
```

```
>>> string.count('s')           >>> lst.count('s')           # 4
```

```
>>> string.find('p')           >>> lst.find('p')           # 8
```

```
>>> string.index('i')          >>> lst.index('i')           # 1
```

```
>>> 'k' in string              >>> 'k' in lst              # False
```

Without using Python built in methods, how can we search for an element in a sequence?

LINEAR SEARCH ALGORITHM

- Sequential search algorithm
 - go through every single element in the sequence
- Order of Growth : $O(n)$

LINEAR SEARCH ALGORITHM – FIND() [BOOLEAN]

```
def find(seq, item):  
    for ele in seq:  
        if ele == item:  
            return True  
    return False
```

```
string = 'mississippi'  
>>> print(find(string, 's'))
```

```
def find(seq, item):
```

Recursive

```
string = 'mississippi'  
>>> print(find(string, 's'))
```

LINEAR SEARCH ALGORITHM – INDEX()

```
def index(seq, item):  
    for i in range(len(seq)):  
        if seq[i] == item:  
            return i  
    return None
```

```
string = 'mississippi'  
>>> print(index(string, 'a'))
```

```
def index(seq, item, i):
```

Recursive

```
string = 'mississippi'  
>>> print(index(string, 'a', 0))
```

LINEAR SEARCH ALGORITHM – COUNT()

```
def count(seq, item):  
    c = 0  
    for ele in seq:  
        if ele == item:  
            c += 1  
    return c  
string = 'mississippi'  
>>> print(count(string, 's'))
```

```
def count(seq, item, c):
```

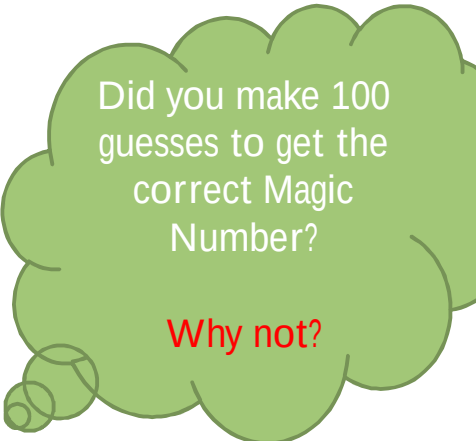
Recursive

```
string = 'mississippi'  
>>> print(count(string, 's', 0))
```

UNPLUGGED ACTIVITY – THE GUESSING GAME

- A Magic Number between 0 to 100 has been random selected
- Make a guess, choose a value
- If your value you have chosen is larger than the Magic Number, then guess with a smaller value next time; if your guess is smaller than the Magic Number, then you need to choose a larger value next time
- Keep guessing until you get the Magic Number
- How many guesses did you make before getting the correct Magic Number?

Run the program code `Guessing Game.py`



Did you make 100 guesses to get the correct Magic Number?

Why not?

BINARY SEARCH ALGORITHM

- Binary search only works with a **sorted** sequence
- "divide and conquer"
- Order of Growth : $O(\log n)$

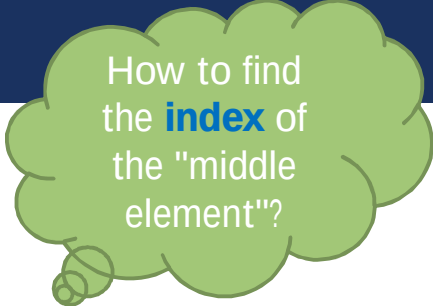
Why is the Order of Growth $O(\log n)$?

<https://hackernoon.com/what-does-the-time-complexity-o-log-n-actually-mean-45f94bb5bfbf>

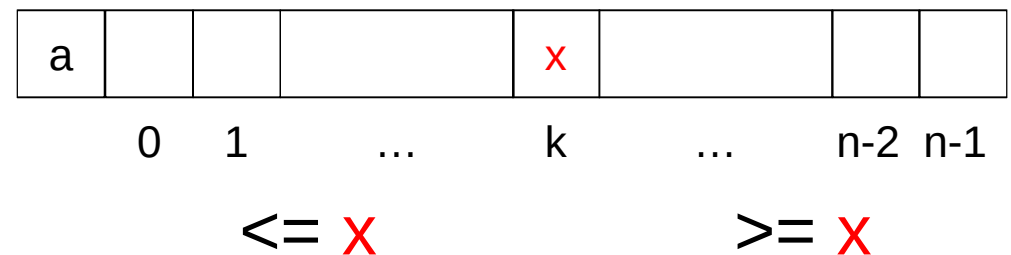
BINARY SEARCH ALGORITHM

For a sorted sequence (eg ascending order) :

- Find the index of the "*middle element*".
- If it is the thing we are looking for, the key, return True.
- If our key is smaller than the middle element, repeat search on the left of the list.
Else, repeat search on the right of the list.



How to find the **index** of the "middle element"?



BINARY SEARCH ALGORITHM

`seq = [5,9,12,18,25,34,85,100,123,345]`

`low = 0`

`high = len(seq) - 1 = 9`

`mid = (low + high) // 2` `# mid = 4`

`mid` is the index of the middle element

`middle element : seq[mid]` `# seq[4] = 25`

Left hand side :

`low = low`

`high = mid - 1`

Right hand side :

`low = mid + 1`

`high = high`

PERFORM BINARY SEARCH USING A TRACE TABLE

- Now let's try to use a trace table to help us organize the flow :

LH

Seq = [0 1 2 3 4 5 6 7 8 9]

5 9 12 18 25 34 85 100 123 345]

KEY = 11

Low	High	Low <= High	Mid	Seq[Mid]	KEY == Seq[Mid]	KEY < Seq[Mid]
0	9					

Left hand side :

low = low

high = mid -1

Right hand side :

low = mid + 1

high = high

BINARY SEARCH ALGORITHM (ITERATIVE)

```
def BinarySearch(seq, item):
```

Iterative

Repeat the following when low <= high:

- $mid = (low + high) // 2$
- check if $seq[mid] == item$; if yes, return True
- if $seq[mid] > item$, search LHS, else search RHS

Note: LHS: low to mid-1
RHS: mid+1 to high

```
lst = [5, 9, 12, 18, 25, 34, 85, 100, 123, 345]  
>>> BinarySearch(lst, 11)
```

BINARY SEARCH ALGORITHM (RECURSIVE)

```
def Search(seq, lo, hi, item):
```

Recursive

```
lst = [5, 9, 12, 18, 25, 34, 85, 100, 123, 345]
>>> Search(lst, 0, 9, 11)
```

Write the code for a helper function
Search(seq, lo, hi, item)

if lo > hi, return False

else: (ie lo ≤ hi)

- mid = (lo + hi) // 2
- check if seq[mid] == item; if yes, return True
- if seq[mid] > item, search LHS, else search RHS

Note: LHS: low to mid-1

RHS: mid+1 to high

BINARY SEARCH ALGORITHM (RECURSIVE)

```
def BinarySearch(seq, item):
```

Recursive
(con't)

Write the code for BinarySearch(seq, item) using the helper function Search(seq, lo, hi, item)

low = 0

high = len(seq) - 1

call the helper function Search()

```
lst = [5, 9, 12, 18, 25, 34, 85, 100, 123, 345]
```

```
>>> BinarySearch(lst, 11)
```

